

Contents

Implementation Plan: Spatially-Varying Driver — Pre-Cluster Unit-Test Gate	1
Overview	1
Constraints	1
Phase 0: Test infrastructure foundations	3
Phase 1: Schema + parser unit tests (QD §1 + DYN §0)	5
Phase 2: Per-DOF resolver tests (QD §1 + DYN §0)	7
Phase 3: Velocity + stress bundle unit tests (QD §2-3 + DYN §3)	8
Phase 4: Operator-accessor regression tests (QD §5)	10
Phase 5: Fault-flux setter tests (DYN §1) + Per-QP impedance tests (DYN §2)	12
Phase 6: Heterogeneous-material plan regression tests (cross-plan gate)	14
Phase 7: Driver --dry-run smoke tests (QD §4 + DYN §4)	15
Phase 8: Restart round-trip tests (QD §4 + DYN §4)	16
Phase 9: Local end-to-end smoke (QD + DYN)	17
Phase 10: Pre-cluster gate — single umbrella + checklist	19
Phase 11: Pre-existing-test regression matrix (continuous gate)	20
Testing Strategy	21
Risk Assessment	22
Out of Scope for This Plan	23

Implementation Plan: Spatially-Varying Driver — Pre-Cluster Unit-Test Gate

Date: 2026-05-17 Companion plans: `spatial_quasi_dynamic_plan.md`, `spatial_dynamic_rupture_plan.md`, `heterogeneous_material_plan.md`.

Overview

A consolidated, local-only test plan that gates submission of any Frontera job for `seas_spatial_qd_driver` / `seas_spatial_dyn_driver`. The plan defines synthetic fixtures (small enough to commit), unit tests for every phase of both driver plans, regression tests that pin bit-exactness for existing BP5/TPV paths, MPI tests at $np \in \{1, 2, 4\}$ on a laptop, and a single make-target `make test-spatial-precluster` that prints PASS / FAIL with per-phase counts. The intent: catch every defect that would otherwise burn Frontera credits, with full local runtime under ~10 minutes.

The plan is intentionally testing-first. It does not specify production code beyond what the parent plans already define; everywhere it says “build X”, X is an item that must already be specified by one of the companion plans (cross-referenced inline as `→ QD §X` or `→ DYN §X`). When a test requires an additive change to production code (a friend declaration, a test-only setter), the requirement is called out under “Production-code prerequisites” inside that phase.

Constraints

Test-framework constraints — what cannot change

- Per-file test binary: each `tests/unit/test_<name>.cpp` produces one executable `seas_test_<name>` driven by an umbrella `make test-<name>`. This matches every existing seas miniapp test. No GoogleTest, no Catch2, no per-file framework switch.

- **TEST_ASSERT(condition, message)** macro pattern with a manual num_tests/num_passed/num_failed counter and a non-zero main() return on any failure. Matches tests/unit/test_safs_mode_wiring.cpp and every other seas test. The macro must be defined inside the test file (no shared header) so each test stays self-contained.
- **SEAS_POST_LINK_DEDUP_RPATH** convention (Makefile R-005 from REVIEW.md): every new link rule must end with \$(SEAS_POST_LINK_DEDUP_RPATH) to avoid macOS 26 duplicate-LC_RPATH abort.
- Existing tests must remain bit-exact: make test, make test-tpv104, the SAFS-side umbrellas (test-safs-mode-wiring, test-compute-safs-params, etc.) must pass with the same outputs they produce today. Tracked by Phase 5's regression matrix.
- MFEM_VERIFY abort harness: tests that exercise abort paths must use a fork()-based child process so the parent test binary survives the abort and reports PASS/FAIL. A small helper tests/unit/expect_abort.hpp is introduced in Phase 0 and reused.

Dependency constraints

- **extern/toml11** (already present; gated by SEAS_USE_TOML in Makefile). Toml-parser tests must run identically when SEAS_USE_TOML=NO by skipping with a SKIP: TOML disabled line that does not count as a failure.
- MFEM build with **MFEM_USE_HDF5=YES** and **MFEM_USE_PETSC=YES**. The QD restart tests require PETSc; if PETSc is disabled they SKIP rather than fail. The ZFP tests SKIP on MFEM_USE_H5Z_ZFP=NO (matching existing seas_test_vtkhdf_zfp pattern).
- Local MPI (the conda mfem-dev env uses OpenMPI 4.x). mpirun -np 4 must complete on a laptop without specifying a node-allocation flag.
- Python 3 + h5py for synthetic-sidecar generators. Both already required by existing tests (e.g., bp5/parametric_study/). Driven by a single helper module tests/fixtures/spatial/build_fixtures.py.

Convention constraints

- All synthetic fixtures live under **tests/fixtures/spatial/** and are COMMITTED (small enough). Mesh + sidecar files together must total ≤ 5 MB so they don't pollute the repo. The build script is committed too so a contributor can regenerate.
- Fixture filenames carry the fixture purpose in the basename, e.g. safs_mini_2lay.msh (mini SAFS-like 2-layer mesh), velocity_sidecar_uniform.h5, velocity_sidecar_2layer.h5, stress_sidecar_planar.h5. Pattern: <dataset_class>_<purpose>.<ext>.
- Test IDs: each test inside a .cpp is tagged T_<PHASE>_<N>_<short_name> (e.g., T_2_1_schema_version_mismatch). The combined ID T_2_1 is greppable from any failure log.
- Runtime budget: every umbrella in Phases 2-9 must finish in < 30 seconds at np=1 and < 90 seconds at np=4. Phase 10 end-to-end smoke is capped at 5 minutes np=4. Phase 11 gate sequencing all phases capped at < 10 minutes total.

Numerical constraints

- Bit-exact regression: when a test re-runs an existing operator on an existing mesh through the new code path with Mode::Constant material + constant friction, the assembled stiffness matrix entries / Mult outputs / ComputeRHS outputs must match the pre-change reference to within 0 ULP (i.e. memcmp-identical), not within a floating-point tolerance.
- Constant-as-heterogeneous parity: when a test runs the new heterogeneous code path with a coefficient that returns a constant everywhere, the output must match the Mode::Constant path to within $1e-10$ relative (slight slack from T.Transform(ip, x) round-trip).
- Heterogeneous correctness (Phase 4-7): when the input changes by Δ , the output must change in the expected direction by the expected magnitude (sensitivity tests, not absolute-value tests).
- Restart bit-exactness (Phase 9): the state vector after restart at step N must equal the state vector after a continuous run at step N to within $||state|_{inf} \cdot 1e-12$.

Phase 0: Test infrastructure foundations

Goal

Provide every reusable building block — abort harness, synthetic-fixture generators, fixture data files — that the subsequent test phases depend on. After this phase the rest of the plan has zero shared-infrastructure work left to do.

Files to Create

- tests/unit/expect_abort.hpp — fork()/wait() wrapper. Header-only.

```
/// Run `fn()` inside a child process; assert the child exits via
/// `std::abort()` (MFEM_VERIFY / MFEM_ABORT path) and the captured
/// stderr contains the expected substring. Returns true on PASS.
/// Aborts the parent with TEST_ASSERT on failure.
///
/// Skips and returns true on Windows or environments without fork().
bool ExpectAbortWithSubstring(std::function<void()> fn,
                             const std::string& expected_substring);
```
- tests/fixtures/spatial/ directory with:
 - build_fixtures.py — Python 3 script (stdlib + h5py + numpy + gmsh) that regenerates every fixture below. Single entry point; python3 build_fixtures.py --all rebuilds everything.
 - safs_mini_2lay.geo — gmsh source for a 5×5×3 km box with a single planar fault at x=0, ~200 tets, 2 material layers via Physical Surface tags.
 - safs_mini_2lay.msh — pre-built gmsh-2.2 ASCII (committed).
 - safs_mini_2lay.npart — METIS 4-partition (committed) so MPI tests are deterministic.
 - velocity_sidecar_uniform.h5 — schema-v1 sidecar, 10×10×10 grid covering the mini-mesh bbox, every voxel (Vp, Vs, ρ) = (5000, 3000, 2700) so heterogeneous paths reduce to constant.
 - velocity_sidecar_2layer.h5 — same grid, sharp Vs step at z=-1500 m (Vs = 1500 above, Vs = 3500 below); Vp scaled to keep Vp/Vs = √3; constant ρ = 2700.
 - stress_sidecar_planar.h5 — schema-v1 stress sidecar over the same grid, planar tau1/tau2/sigma_n with depth-linear σ_n.
 - stress_sidecar_zero_normal_trigger.h5 — small sidecar that, combined with safs_mini_2lay.msh, produces a FaultGeometry::NumZeroNormalFallbacks() > 0. Used to test R-001 SetSAFSMode guard.
 - friction_rate_state_uniform.toml — minimal rate-state TOML (default scalars, no spatial rules) for use by parser tests.
 - friction_slip_weakening_uniform.toml — minimal LSW TOML.
 - friction_rate_state_layered.toml — uses [[spatial]] depth-rule pair (shallow strengthening + seismogenic core).
 - friction_rate_state_invalid_schema_v2.toml — TOML with schema_version = 2 so parser-abort test has a fixture to bite on.
 - friction_rate_state_invalid_a_gt_b.toml — TOML where default a_default > b_default, triggers resolver-validation abort.

Files to Modify

- miniapps/seas/Makefile:
 - Add tests/fixtures/spatial/ to a fixtures-clean and fixtures-rebuild target (calls python3 build_fixtures.py --all).

- Add tests/fixtures/spatial/ to .gitattributes (*.msh *.h5 binary) so git LFS or pack handles them gracefully.
- miniapps/seas/.gitignore: ensure tests/fixtures/spatial/ __pycache__ / and tests/fixtures/spatia (build-time staging) are ignored.

Detailed Requirements

1. **ExpectAbortWithSubstring** implementation (tests/unit/expect_abort.hpp):

- Use pipe() to capture child stderr, fork() to spawn the child, dup2 the pipe write-end to stderr.
- In the child, call fn(); if it returns, _exit(0) (test fails — abort expected).
- Parent: waitpid for child; check WIFSIGNALED(status) && WTERMSIG(status) == SIGABRT (Linux) OR WIFEXITED(status) && WEXITSTATUS(status) != 0 (some libstdc++ abort paths exit non-zero rather than raise SIGABRT). Read pipe; verify expected_substring appears.
- Tolerate the case where the child's abort message reaches stdout instead of stderr (some MFEM versions); capture both.
- Header-only via inline definitions.

2. **build_fixtures.py** requirements:

- One main() with --all, --mesh, --velocity, --stress, --friction selectors.
- Idempotent: re-running with --all produces byte-identical files.
- Each artefact prints [BUILD] + path + size in bytes; aborts on any tool failure.
- Honors --out-dir (default tests/fixtures/spatial/); useful for staging in CI.

3. **safs_mini_2lay.geo** design:

- Box: $x \in [-5000, 5000]$, $y \in [-2500, 2500]$, $z \in [-3000, 0]$.
- Fault surface: Physical Surface("fault", 101) = {fault_surface_id}.
- Other surfaces tagged 102 (top), 103 (bottom), 104 (sides) — mirrors safs_fault_box_nwcut.geo.
- Two Physical Volumes (101 "shallow", 102 "deep") split at $z = -1500$ m, even though the mesh uses a single material — the tag is for spatial-rule tests in Phase 3.
- Element size $lc = 800$ m so the mesh fits ~200 tets and serialises < 100 KB.

4. Synthetic sidecar HDF5 schema: matches the existing schema-v1 used by DataField3D. Generator must use the same group / dataset names so DataField3D(sidecar_path) loads the synthetic file without modification.

5. **stress_sidecar_zero_normal_trigger.h5** requirements: the sidecar's spatial domain is set to MISS one corner of the mini-mesh (specifically, the +x +y +z corner) by ~50 m. This forces at least one fault DOF to fall outside the sidecar's interpolable region, which in turn forces FaultGeometry::GetFaultDOFBasis to return a zero column for that DOF, which surfaces as a NumZeroNormalFallbacks() = 1 post-ctor. Verify the trigger is reliable by running the BP5 ctor on this fixture in Phase 5 acceptance test T_5_R001_fixture_works.

Interfaces

- bool ExpectAbortWithSubstring(std::function<void()>, const std::string&).
- tests/fixtures/spatial/build_fixtures.py --all (rebuilds everything).
- tests/fixtures/spatial/ (data directory).

Edge Cases to Handle

- **fork()** not available (cross-build for Windows in CI) — ExpectAbortWithSubstring returns true with a SKIP message printed to stdout.

- **build_fixtures.py** partial output on failure — generator writes to .tmp paths and atomically re-names on success; on error, removes .tmp files and exits non-zero with the offending tool's stderr.
- mismatched gmsh version — generator pins gmsh -format msh22 (ASCII 2.2) so MFEM's Mesh(path) reads without warnings.

Acceptance Criteria

- ☐ python3 tests/fixtures/spatial/build_fixtures.py --all succeeds in the pythonenv conda env, produces all 11 fixture files listed in Files to Create.
- ☐ Total size of tests/fixtures/spatial/ ≤ 5 MB.
- ☐ Rerunning build_fixtures.py --all produces byte-identical files (idempotency).
- ☐ seas_test_expect_abort_smoke (Phase 0's own unit test, ~30 lines) confirms ExpectAbortWithSubstring catches both std::abort() and MFEM_ABORT paths with the expected substring.

Dependencies

- Depends on: nothing.
- Required by: every subsequent phase.

Phase 1: Schema + parser unit tests (QD §1 + DYN §0)

Goal

Pin every validation rule of SpatialFrictionConfig so a malformed TOML cannot reach the driver.

Production-code prerequisites

- QD §1 SpatialFrictionConfig, LoadSpatialFrictionConfig, SpatialTimeParser, SpatialFrictionResolver declarations.

Files to Create

- tests/unit/test_spatial_friction_config.cpp — TOML round-trip + validation.
- tests/unit/test_spatial_time_parser.cpp — "300yr" / "5Ma" / "0.01s" parsing.

Detailed Test Cases

test_spatial_friction_config.cpp (12 tests):

ID	Name	Mechanism
T_1_1	uniform_rate_state_loads	LoadSpatialFrictionConfig(fixture/spat returns law == RateState, all scalar defaults read
T_1_2	uniform_slip_weakening_loads	Same for friction_slip_weakening_uniform.toml, law == SlipWeakening
T_1_3	layered_rate_state_loads	friction_rate_state_layered.toml returns 2 SpatialRules in rate_state->spatial

ID	Name	Mechanism
T_1_4	schema_version_mismatch_aborts	friction_rate_state_invalid_schema_v2.t → ExpectAbortWithSub- string(..., "schema_version")
T_1_5	unknown_law_aborts	inline TOML with law = "garbage" → abort with "unknown law"
T_1_6	both_blocks_present_aborts	TOML with both [friction.rate_state] and [friction.slip_weakening] → abort with "both ... present"
T_1_7	missing_block_aborts	law = "rate_state" but no [friction.rate_state] table → abort with "missing [friction.rate_state]"
T_1_8	unknown_key_in_rate_state_aborts	extra key [friction.rate_state] mystery = 1.0 → abort with "unknown key: mystery"
T_1_9	a_gt_b_default_aborts	friction_rate_state_invalid_a_gt_b.toml → abort with "a < b"
T_1_10	negative_Dc_aborts	inline TOML with Dc_default = -1.0 → abort with "Dc > 0"
T_1_11	pore_pressure_negative_min_aborts	[pore_pressure] min_sigma_n_pa = -1.0 → abort with "min_sigma_n_pa >= 0"
T_1_12	toml_disabled_skip	when SEAS_USE_TOML is NO, every test in this file prints SKIP: TOML disabled and exits 0

test_spatial_time_parser.cpp (8 tests):

ID	Name	Input	Expected
T_1_20	bare_seconds	"1.5"	1.5 s
T_1_21	seconds_suffix	"100s"	100.0 s
T_1_22	years	"1yr"	31_557_600.0 s (365.25 × 86400)
T_1_23	ka	"2ka"	6.3115e10 s
T_1_24	Ma	"0.5Ma"	1.577e13 s
T_1_25	scientific	"1.2e6s"	1.2e6 s
T_1_26	unknown_suffix	"300week"	abort with "unknown time suffix: week"
T_1_27	nan_input	"nan"	abort with "unparseable time"

Acceptance Criteria

- ☐ Both binaries build and run via make test-spatial-friction-config and make test-spatial-time-parser.

- 12 + 8 = 20 tests all PASS at np=1.
- Runtime < 5 s combined.

Dependencies

- Depends on: Phase 0 (fixtures, ExpectAbortWithSubstring); QD §1 implementation.
- Required by: Phase 2.

Phase 2: Per-DOF resolver tests (QD §1 + DYN §0)

Goal

Pin the spatial-rule application order, last-match-wins semantics, and eta-from-material auto path.

Production-code prerequisites

- QD §1 `SpatialFrictionResolver::ResolveRateState` and `ResolveSlipWeakening` implementations.

Files to Create

- `tests/unit/test_spatial_friction_resolver.cpp`.

Detailed Test Cases (12 tests):

Synthetic fixture for all tests in this file: a hand-built `Vector dof_coords_3d` of 6 fault DOFs at known (x, y, z) , with `dof_to_attr = [101]*6` (single fault region).

ID	Rule set	Assertion
T_2_1	no spatial rules	every output equals the corresponding <code>*_default</code> exactly
T_2_2	one depth rule overrides a for $z \in [-3000, -1000]$	only DOFs inside the depth range have new a; others keep default
T_2_3	overlapping depth rules (last wins)	DOF inside both rules takes the second rule's value
T_2_4	box rule with x, y, z bounds	only DOFs inside the box are overridden
T_2_5	region_attribute rule	DOFs whose <code>dof_to_attr</code> matches get the override; others don't
T_2_6	NaN override field	NaN means "don't override" — partial-field overrides leave other fields unchanged
T_2_7	eta_auto via <code>MaterialField::MakeConstant(λ, μ, ρ)</code>	every DOF's $\eta == 0.5 * \sqrt{\mu * \rho}$ exactly
T_2_8	eta_default scalar	<code>eta_auto_from_material = false</code> , all DOFs return <code>cfg.eta_default</code>

ID	Rule set	Assertion
T_2_9	pore-pressure depth gradient	$\sigma_n^{\text{eff}}(z=-2000) = \sigma_n - (P_p^{\text{pa}} + P_p^{\text{grad}} * 2000)$, exact
T_2_10	pore-pressure clamp	result clamped at $\min_sigma_n_pa$
T_2_11	validation: post-resolve a > b aborts	construct a rule that drives a > b at one DOF → abort with "a < b at DOF N" (via ExpectAbortWithSubstring)
T_2_12	ResolveSlipWeakening: depth-barrier rule	DOFs at $z < -15$ km have $\mu_s = 1.0e6$ (barrier sentinel)

Acceptance Criteria

- ☐ 12 / 12 PASS.
- ☐ Runtime < 3 s.

Dependencies

- Depends on: Phase 1.
- Required by: Phase 8.

Phase 3: Velocity + stress bundle unit tests (QD §2-3 + DYN §3)

Goal

Pin sidecar-path resolution, ContainsBBox pre-flight, ComputeSAFSPParams round-trip, and the R-001 zero-normal guard.

Production-code prerequisites

- QD §2 LoadSpatialVelocityBundle; SpatialVelocitySpec resolver.
- QD §3 ApplySpatialStressSidecar (uses existing StressField3D + ComputeSAFSPParams).
- The R-001 SetSAFSMode guard from the most recent REVIEW.md cycle (already merged).

Files to Create

- tests/unit/test_spatial_velocity_bundle.cpp.
- tests/unit/test_spatial_stress_bundle.cpp.

Detailed Test Cases

test_spatial_velocity_bundle.cpp (6 tests):

ID	Name	Mechanism
T_3_1	resolve_cvmh_path	ResolveSpatialVelocitySidecarPath({CVMH "1000m_lcfar3000"}, "/tmp/data") returns /tmp/data/velocity/results/cvmh/velocity (string equality)
T_3_2	resolve_override_bypasses	non-empty override_path returned verbatim
T_3_3	missing_file_aborts	model resolves to nonexistent path → abort with "sidecar not found: <path>"
T_3_4	uniform_sidecar_loads	LoadSpatialVelocityBundle on veloc- ity_sidecar_uniform.h5 + safs_mini_2lay.msh returns bundle with mode == Coefficient, all three coef pointers non-null
T_3_5	mesh_outside_bbox_aborts	a synthetic sidecar whose bbox excludes the mini-mesh → abort with the existing FieldProjec- tor::AbortContainmentFailure table
T_3_6	lifetime_contract	bundle in inner scope, MaterialField returned by MakeMaterialField() outside scope → accessing the MaterialField's lambda_coef → Eval(...) after bundle destruction either ASAN-traps or SEGFAULTs (run under ASAN_OPTIONS=halt_on_error=1 if available; SKIP otherwise)

test_spatial_stress_bundle.cpp (5 tests):

ID	Name	Mechanism
T_3_10	planar_stress_loads_no_fallback	ApplySpatialStressSidecar({stress_sidecar succeeds on safs_mini_2lay.msh; geom.HasSAFSParams() == true; geom.NumZeroNormalFallbacks() == 0
T_3_11	sigma_n_per_dof_populated	after T_3_10, geom.sigma_n_per_dof().Size() == geom.NumFaultDOFs(), every entry > 0
T_3_12	tau_pre_per_dof_populated	analogous; size 2 * num_fault_dofs, no NaN

ID	Name	Mechanism
T_3_13	zero_normal_trigger_aborts	use stress_sidecar_zero_normal_trigger.h5 → ApplySpatialStressSidecar aborts BEFORE calling ComputeSAFSParams with substring "NumZeroNormalFallbacks"
T_3_14	stress_sidecar_missing_aborts	path = /nope.h5 → abort with "sidecar not found"

Acceptance Criteria

- ☐ 6 + 5 = 11 PASS.
- ☐ Runtime < 8 s combined (HDF5 loads dominate).

Dependencies

- Depends on: Phase 0 (fixtures); QD §2-3 implementation.
- Required by: Phase 8.

Phase 4: Operator-accessor regression tests (QD §5)

Goal

Pin bit-exactness of every existing operator path that the new additive accessors might touch.

Production-code prerequisites

- QD §5 ElasticityDomainOperator::GetFaultElemOwners() accessor.
- QD §5 RateStateFaultOperator::SetPerD0FRateStateParams(...) setter.

Files to Create

- tests/unit/test_per_dof_rate_state_params.cpp (3 tests).
- tests/unit/test_get_fault_elem_owners.cpp (2 tests).

Detailed Test Cases

test_per_dof_rate_state_params.cpp (3 tests):

ID	Name	Mechanism
T_4_1	default_off_bit_exact	Build RateStateFaultOperator on safs_mini_2lay.msh BP5 path; call PreInit + ComputeRHS; record output; rebuild; do NOT call SetPerD0FRateStateParams; call PreInit + ComputeRHS again; assert memcmp of the two rate Vectors == 0
T_4_2	setter_with_scalar_match_bit_exact	Call SetPerD0FRateStateParams(a_vec, b_vec, ...) where every vector entry equals the corresponding bp5_params_scalar; assert ComputeRHS output memcmp-identical to T_4_1 baseline
T_4_3	one_dof_a_perturbed_changes_output	Same as T_4_2 but a_vec(2) = bp5_params_.a * 1.10; assert rate(2 * StatePerNode + ...) differs by the expected slip-rate-derivative ratio (analytical: dV/da from the steady-state friction relation), tolerance 1e-3 * baseline

test_get_fault_elem_owners.cpp (2 tests):

ID	Name	Mechanism
T_4_10	bp5_mesh_owners_nonempty	Construct ElasticityDomainOperator on safs_mini_2lay.msh; GetFaultElemOwners().Size() == GetNumFaultD0Fs(); every owner index is in [0, mesh.GetNE())
T_4_11	antiplane_owners_empty	Construct AntiplaneDomainOperator on bp2_full.msh (a fixture already in bp2/mesh/); GetFaultElemOwners().Size() == 0

Acceptance Criteria

- ☐ 3 + 2 = 5 PASS.
- ☐ make test-safs-mode-wiring, make test-compute-safs-params, make test-bp5-integration continue to pass bit-identically (regression).
- ☐ Runtime < 5 s combined.

Dependencies

- Depends on: Phase 0; QD §5 implementation.
 - Required by: Phase 8.
-

Phase 5: Fault-flux setter tests (DYN §1) + Per-QP impedance tests (DYN §2)

Goal

Pin the dynamic-rupture-only setters before the driver wires them.

Production-code prerequisites

- DYN §1 `FaultFaceFlux::InitializeFromSpatialStress` and `InitializeFromSpatialStressRateState`.
- DYN §2 `FaultFaceFlux::InitializeImpedancesPerQP`.

Files to Create

- `tests/unit/test_spatial_dyn_fault_setup_lsw.cpp` (5 tests).
- `tests/unit/test_spatial_dyn_fault_setup_rs.cpp` (4 tests).
- `tests/unit/test_fault_face_flux_per_qp_impedance.cpp` (4 tests).

Detailed Test Cases

test_spatial_dyn_fault_setup_lsw.cpp (5 tests):

ID	Name	Mechanism
T_5_1	tpv205_bit_exact	Construct <code>FaultFaceFlux</code> as in the existing TPV205 path; do NOT call <code>InitializeFromSpatialStress</code> ; <code>dof_data_entries</code> match an existing TPV205 reference byte-for-byte
T_5_2	spatial_setter_writes_dof_data	After <code>InitializeFromSpatialStress(tau_pre, sigma_n, mu_s, mu_d, d_c, {})</code> , every <code>dof_data[i]</code> has the supplied values; <code>spatial_initialized_ == true</code>
T_5_3	size_mismatch_aborts	<code>tau_pre.Size() == 3 * num_fault_dofs</code> (wrong) → abort with "tau_pre size"
T_5_4	double_init_aborts	Call setter twice → second call aborts with "InitializeFromSpatialStress called twice"

ID	Name	Mechanism
T_5_5	barrier_dof_passthrough	LSW $\mu_s = 1.0e6$ at DOF 2; setter writes; existing EvaluateADER_LSW treats DOF 2 as barrier (downstream verified by integration test in Phase 8)

test_spatial_dyn_fault_setup_rs.cpp (4 tests):

ID	Name	Mechanism
T_5_10	rs_setter_writes_dof_data	After InitializeFromSpatialStressRateState(...), RS fields populated, LSW fields stay zero
T_5_11	mixed_population_misuse_guard	Call BOTH InitializeFromSpatialStress and InitializeFromSpatialStressRateState → second call aborts (the spatial_initialized_guard fires)
T_5_12	rs_size_mismatch	analog of T_5_3 for RS
T_5_13	rs_zero_a_aborts	a.SetSize(N); a = 0.0; → setter validates a > 0, aborts with "a > 0"

test_fault_face_flux_per_qp_impedance.cpp (4 tests):

ID	Name	Mechanism
T_5_20	non_safs_path_bit_exact	impedances_per_qp_active_ == false by default; existing EvaluateADER_LSW reads scalar eta_p_ / eta_s_ — bit-exact vs. TPV205 reference
T_5_21	constant_parity	Call InitializeImpedancesPerQP(MaterialField::MakeConstant(λ , μ , ρ)); every eta*_per_dof_(i) equals the scalar baseline to 1e-12
T_5_22	layered_sidecar_varies	Use velocity_sidecar_2layer.h5 to build a Coefficient-mode MaterialField; impedances at deep DOFs differ from shallow DOFs by the expected $\sqrt{\mu * \rho}$ ratio

ID	Name	Mechanism
T_5_23	lifetime_independence	Initialise from a MaterialField, destroy the MaterialField, run EvaluateADER_LSW — eta vectors still valid (no use-after-free)

Acceptance Criteria

- ☐ 5 + 4 + 4 = 13 PASS.
- ☐ Existing TPV104/205 regression tests (make test-tpv104) remain bit-identical.
- ☐ Runtime < 10 s combined.

Dependencies

- Depends on: Phase 0, Phase 4; DYN §1-2 implementation; heterogeneous_material_plan.md Phase 1 (MaterialField::Mode::Coefficient).
- Required by: Phase 8.

Phase 6: Heterogeneous-material plan regression tests (cross-plan gate)

Goal

The two driver plans depend on heterogeneous_material_plan.md Phases 1-3. If those phases regress (e.g., GodunovFluxPool memory blow-up or face-neighbour exchange breaks), the SAFS driver runs will burn cluster credits before failing. Pin them locally.

Production-code prerequisites

- heterogeneous_material_plan.md Phases 1, 2, 3 (already specified there; this phase is test-only).

Files to Create

- tests/unit/test_material_field_coefficient_mode.cpp (8 tests covering MaterialField::Mode::Coefficient + EvalAt + MaxCpInElement).
- tests/unit/test_elasticity_operator_coefficient_mode.cpp (5 tests covering EOP constructor parity + layered sidecar correctness).
- tests/unit/test_wave_operator_coefficient_mode.cpp (6 tests including the bi-material averaged-flux test from heterogeneous_material_plan §T-WAVEOP-BIMATERIAL-AVG-FLUX).
- tests/unit/test_godunov_flux_pool.cpp (4 tests covering uniqueness-map dedup + memory budget).

Detailed Test Cases

test_material_field_coefficient_mode.cpp — re-uses the test catalog from heterogeneous_material_plan.md Testing Strategy §Phase 1 (T-5-4 ... T-5-7) plus 4 new tests on EvalAt semantics and MaxCpInElement Coefficient-path correctness.

test_elasticity_operator_coefficient_mode.cpp — re-uses T-EOP-CONST-REGRESSION, T-EOP-COEFF-CONST-PARITY, T-EOP-COEFF-LAYERED, T-EOP-MATERIAL-LIFETIME from heterogeneous_material_plan.md Phase 2.

test_wave_operator_coefficient_mode.cpp — re-uses T-WAVEOP-CONST-PARITY, T-WAVEOP-COEFF-PARITY, T-WAVEOP-CFL-CONST-PARITY, T-WAVEOP-CFL-HETERO, T-WAVEOP-BIMATERIAL-AVG-FLUX, T-WAVEOP-COEFF-CENTROID-PARITY from heterogeneous_material_plan.md Phase 3.

test_godunov_flux_pool.cpp — 4 tests: - T_6_30 mode_constant_single_flux: Mode::Constant pool has NumUniqueFluxes() == 1. - T_6_31 mode_coefficient_dedups_layered: 2-layer sidecar produces NumUniqueFluxes() $\in [2, 4]$ (small slack for the rounded-triple hash). - T_6_32 memory_budget_under_2GB_mini_mesh: on safs_mini_2lay.msh with Mode::Coefficient + velocity_sidecar_2layer.h5, NumUniqueFluxes() * kGodunovFluxBytes $\leq 2.0e9$ (assert formula matches the heterogeneous_material plan's R-011 budget formula). - T_6_33 nbr_exchange_smoke_np2: mpirun -np 2 seas_test_godunov_flux_pool confirms AtNbr(...) returns non-default flux on shared-face DOFs of safs_mini_2lay.msh (uses the committed 4-partition + reuses np=2 partition by combining METIS parts).

Acceptance Criteria

- ☐ 8 + 5 + 6 + 4 = 23 PASS at np=1; 1 PASS at np=2 (T_6_33).
- ☐ Runtime < 30 s combined (np=1 batch).
- ☐ Memory budget assertion in T_6_32 passes; if it fails, the heterogeneous_material plan's mitigation (loosen dedup_sig_figs to 4) must be applied BEFORE proceeding.

Dependencies

- Depends on: Phase 0; heterogeneous_material_plan.md Phases 1-3 implementation.
 - Required by: Phase 8 (driver assumes all of this passes).
-

Phase 7: Driver **--dry-run** smoke tests (QD §4 + DYN §4)

Goal

Confirm both drivers reach the time-loop boundary without errors on a tiny fixture, at $np \in \{1, 4\}$.

Production-code prerequisites

- QD §4 seas_spatial_qd_driver with --dry-run.
- DYN §4 seas_spatial_dyn_driver with --dry-run.

Files to Create

- tests/unit/test_spatial_qd_driver_dry_run.cpp — popen-based driver invocation.
- tests/unit/test_spatial_dyn_driver_dry_run.cpp — same for dynamic.
- tests/fixtures/spatial/spatial_qd_config_mini.toml — full TOML pointing at all the Phase-0 fixtures (mini mesh, uniform sidecar, planar stress, rate-state uniform friction).
- tests/fixtures/spatial/spatial_dyn_config_mini.toml — LSW counterpart.

Detailed Test Cases

test_spatial_qd_driver_dry_run.cpp (6 tests):

ID	Name	Mechanism
T_7_1	qd_dry_run_np1_exits_zero	popen("./seas_spatial_qd_driver --config <toml> --dry-run"); assert exit 0
T_7_2	qd_dry_run_np1_prints_derived	Output contains [DERIVED] L_nuc = ... line
T_7_3	qd_dry_run_equilibrium_passes	Output contains [INIT] equilibrium: max F / V_init_max = X < 1e-6
T_7_4	qd_dry_run_np4_exits_zero	mpirun -np 4 ./seas_spatial_qd_driver ... (skips if MPI not available)
T_7_5	qd_dry_run_no_output_files	After dry-run, no volume.vtkhdf exists in the output dir (dry-run skips time loop AND output init)
T_7_6	qd_dry_run_missing_velocity_aborts	Override velocity-sidecar path to nonexistent file → driver exits non-zero with the expected error substring

test_spatial_dyn_driver_dry_run.cpp (6 tests): mirror structure with `--ader-order 2, --cfl 0.5, --mixed-flux none`; T_7_13 asserts [DERIVED] CFL $\Delta t \in [..., ...]$ rather than L_nuc.

Acceptance Criteria

- ☐ 6 + 6 = 12 PASS at np=1; 2 PASS at np=4.
- ☐ Runtime < 30 s combined (driver startup + ParMesh distribute + sidecar loads dominate).

Dependencies

- Depends on: Phases 1-6.
- Required by: Phase 8.

Phase 8: Restart round-trip tests (QD §4 + DYN §4)

Goal

Pin V2 PETSc-TS restart for QD and V1 TPV104 restart for dynamic with driver_tag extension.

Production-code prerequisites

- QD §4 driver with `--restart`.
- DYN §4 driver with `--restart` and driver_tag schema field.

Files to Create

- tests/unit/test_spatial_qd_restart_round_trip.cpp (3 tests; skips on MFEM_USE_PETSC=NO).
- tests/unit/test_spatial_dyn_restart_round_trip.cpp (3 tests).

Detailed Test Cases

test_spatial_qd_restart_round_trip.cpp:

ID	Name	Mechanism
T_8_1	qd_v2_round_trip_np1	Run driver to step 50 with <code>--checkpoint-every 50</code> (writes one checkpoint at step 50, exits at step 100); record state at step 100. Restart from step-50 checkpoint with <code>--tfinal</code> extended; run to step 100. Assert <code> state_continuous - state_restart _inf ≤ 1e-12 * state _inf</code> .
T_8_2	qd_v2_round_trip_np4	Same at np=4; skips if MPI unavailable.
T_8_3	qd_v2_no_checkpoint_aborts	<code>--restart</code> pointing at nonexistent file → exit non-zero with "checkpoint not found".

test_spatial_dyn_restart_round_trip.cpp: mirror structure with V1 schema; plus T_8_13 specific to driver_tag:

ID	Name	Mechanism
T_8_13	dyn_v1_wrong_driver_tag_aborts	Hand-edit a checkpoint to set <code>driver_tag = "tpv104"</code> ; restart with <code>seas_spatial_dyn_driver --restart ...</code> → abort with "driver_tag mismatch".

Acceptance Criteria

- ☐ 3 + 3 = 6 PASS at np=1; 2 PASS at np=4 (T_8_2 and DYN equivalent).
- ☐ Runtime < 60 s combined (each round-trip runs 100 short steps twice).

Dependencies

- Depends on: Phase 7.
- Required by: Phase 11.

Phase 9: Local end-to-end smoke (QD + DYN)

Goal

Run a real (non-dry-run) short simulation of each driver on the mini fixture at np=4 and verify the output files are non-empty and the slip field has plausible behaviour.

Production-code prerequisites

- Both drivers fully wired (Phases 1-8 of this plan all PASS).

Files to Create

- tests/unit/test_spatial_qd_e2e_mini.cpp — 5 tests.
- tests/unit/test_spatial_dyn_e2e_mini.cpp — 5 tests.
- tests/scripts/spatial_qd_e2e_verify.py / spatial_dyn_e2e_verify.py — small h5py inspection scripts called from the C++ test via popen.

Detailed Test Cases

test_spatial_qd_e2e_mini.cpp (5 tests; runs `mpirun -np 4 seas_spatial_qd_driver` with `--tfinal "0.001yr"`):

ID	Name	Assertion
T_9_1	qd_e2e_exits_zero	driver exits 0
T_9_2	qd_e2e_volume_vtkhdf_nonempty	<out>/volume.vtkhdf exists and is > 1 KB
T_9_3	qd_e2e_fault_vtkhdf_nonempty	<out>/fault.vtkhdf exists and > 1 KB
T_9_4	qd_e2e_fault_has_at_least_two_snapshots	posts walks fault.vtkhdf cycles, asserts >= 2 snapshots in 0.001 yr
T_9_5	qd_e2e_slip_rate_in_plausible_range	min \ V_max \ over snapshots $\geq 1e-15$, max $\leq 1e1$ m/s (sanity, not physics)

test_spatial_dyn_e2e_mini.cpp (5 tests; `mpirun -np 4 seas_spatial_dyn_driver` with `--tfinal "0.05s"` and LSW config):

Same structure; T_9_13 asserts bulk.vtkhdf is also non-empty (DYN has two volume collections); T_9_15 asserts max slip-rate exceeds $1e-9$ m/s somewhere in the run (proves the rupture solver is actually computing).

Acceptance Criteria

- ☐ 5 + 5 = 10 PASS at np=4.
- ☐ Runtime: QD ≤ 90 s, DYN ≤ 180 s.
- ☐ Total output volume ≤ 50 MB (mini mesh keeps it tiny).

Dependencies

- Depends on: Phases 1-8.
- Required by: Phase 10.

Phase 10: Pre-cluster gate — single umbrella + checklist

Goal

A single make target `make test-spatial-precluster` that runs every test from Phases 1-9 and prints a PASS/FAIL summary. Submitting any Frontera sbatch BEFORE this target shows ALL PASS is a process violation.

Files to Create

- `miniapps/seas/Makefile`: new umbrella

```
test-spatial-precluster: \  
    test-spatial-friction-config \  
    test-spatial-time-parser \  
    test-spatial-friction-resolver \  
    test-spatial-velocity-bundle \  
    test-spatial-stress-bundle \  
    test-per-dof-rate-state-params \  
    test-get-fault-elem-owners \  
    test-spatial-dyn-fault-setup-lsw \  
    test-spatial-dyn-fault-setup-rs \  
    test-fault-face-flux-per-qp-impedance \  
    test-material-field-coefficient-mode \  
    test-elasticity-operator-coefficient-mode \  
    test-wave-operator-coefficient-mode \  
    test-godunov-flux-pool \  
    test-spatial-qd-driver-dry-run \  
    test-spatial-dyn-driver-dry-run \  
    test-spatial-qd-restart-round-trip \  
    test-spatial-dyn-restart-round-trip \  
    test-spatial-qd-e2e-mini \  
    test-spatial-dyn-e2e-mini  
    @echo "===== "  
    @echo "  spatial pre-cluster gate: ALL PHASES PASS "  
    @echo "===== "
```

- `tests/scripts/spatial_precluster_gate.sh` — wraps the umbrella + emits a one-line mark-down report (PASS / FAIL + per-umbrella counts + total wall time) that the user pastes into the sbatch commit message.
- `safs/project_7.0_alternative/document/spatial_precluster_checklist.md` — printable checklist:

```
Before submitting `safs_*_smoke_8N_400r_dev_2hr.sbatch`:  
[ ] `make test-spatial-precluster` → ALL PASS  
[ ] `make test` (existing umbrella) → no new failures vs. main  
[ ] `make test-tpv104` → 55/55 PASS (proves no regression to dynamic stack)  
[ ] `make test-safs-mode-wiring` → 8/8 PASS (proves R-001 guard intact)  
[ ] `git status` clean (no uncommitted Makefile or fixture changes)  
[ ] Frontera disk quota check: `du -sh $WORK`  
[ ] Frontera SU budget check: `/usr/local/etc/taccinfo`  
Then: `sbatch jobs/safs/spatial_qd_smoke_8N_400r_dev_2hr_safs.sbatch`
```

Detailed Requirements

1. Umbrella ordering: tests run in dependency order (Phase 1 first). Make's `-j` parallelism is disabled (`.NOTPARALLEL: test-spatial-precluster`) to keep error output readable.
2. Single-line PASS report: after every umbrella, `spatial_precluster_gate.sh` greps for passed, / failed, patterns and prints a one-line summary; aborts the gate on the first failure.
3. Wall-time budget assertion: gate aborts if any single umbrella exceeds $3\times$ its expected runtime (defined in Phases 1-9), printing the violating umbrella and the threshold.
4. The checklist file is the SOURCE OF TRUTH: drivers' sbatch headers contain a one-line comment `# Pre-flight: see spatial_precluster_checklist.md` so anyone running the job hits the requirement.

Edge Cases to Handle

- MPI not available on the dev box → mpi-dependent tests print `SKIP: MPI not available (exit 0)`; gate-script counts these as passes but tags the final report (`reduced gate`).
- PETSc not available → QD restart tests skip; gate report tagged (`no PETSc; QD restart skipped`).
- ZFP not enabled → no impact on this gate (no test in Phases 1-9 requires ZFP; the `fault.vtkhdf` in Phase 9 uses uncompressed defaults).

Acceptance Criteria

- ☐ `make test-spatial-precluster` runs sequentially in ≤ 10 minutes wall on the laptop (clang on darwin) and prints `ALL PHASES PASS`.
- ☐ `tests/scripts/spatial_precluster_gate.sh` prints a markdown report under 25 lines, machine-readable enough for the user to paste into a sbatch commit message.
- ☐ After this phase ships, no Frontera sbatch may be submitted for the spatial drivers without an attached gate-report showing ALL PASS — enforced by review in PR description.

Dependencies

- Depends on: Phases 1-9.
 - Required by: cluster runs (everything in `spatial*_plan.md` Phase 6 sbatch lists).
-

Phase 11: Pre-existing-test regression matrix (continuous gate)

Goal

Ensure the test-plan implementation does not regress any existing passing umbrella. This phase is the SAFETY NET — every commit during Phases 0-10 reruns this matrix before merging.

Files to Create

- `tests/scripts/spatial_no_regression_matrix.sh` — runs the matrix below and diffs the per-umbrella PASS counts against a committed baseline.

Matrix (status at start of plan, locked in)

Umbrella	Required count	Notes
make test (primary)	19/20 umbrellas PASS per current REVIEW.md sweep	The 4 ValidateFacetBCTables aborts + parallel-test 138s + bp2_serial_smoke.o compile error remain pre-existing Banner-string fix from REVIEW.md R-007 R-001 guard SAFS Phase 6 §5
make test-tpv104	55/55	
make test-safs-mode-wiring	8/8 (incl. new T_66_5)	
make test-compute-safs-params	13/13	velocity projector paraview Phase 2b
make test-data-projection	4/4 (all 4 SAFS Phase 3-5 tests) (per existing umbrella)	
make test-fault-surface-vtkhdf	(per existing umbrella)	BP5 V2 restart
make test-bp5-petsc-ts-restart	(per existing umbrella)	TPV104 V1 restart
make test-tpv104-checkpoint		

Detailed Requirements

1. Baseline file committed at tests/baselines/spatial_no_regression_baseline.txt records the matrix above as machine-readable <umbrella>:<passes>/<total>. Gate compares against this file; any drop in pass count aborts the gate.
2. Allowed deltas: pass count may INCREASE (new tests added); never decrease.
3. Frequency: runs in CI on every PR touching miniapps/seas/; runs locally before every commit that touches files cross-referenced by both spatial plans (e.g. domain/domain_operator.hpp, fault/rate_state_fault.hpp, dynamic/fault_face_flux.cpp).

Acceptance Criteria

- ☐ Baseline file committed and reflects current REVIEW.md sweep result.
- ☐ tests/scripts/spatial_no_regression_matrix.sh runs locally in < 15 minutes (the existing matrix is ~10 minutes).

Dependencies

- Depends on: nothing in this plan (just records existing state).
- Required by: every other phase of this plan (each phase runs the matrix before declaring DONE).

Testing Strategy

This entire document IS the testing strategy. The structure:

1. Phase 0 builds the infrastructure (fixtures + abort harness).
2. Phases 1-7 are focused unit tests, each pinning one production unit from the driver plans.
3. Phase 8-9 are integration tests (dry-run + e2e) that exercise the driver end-to-end on a tiny fixture.
4. Phase 10 wraps everything into a single gate target.
5. Phase 11 is the parallel safety net that pins existing passing umbrellas during all the above work.

No production code is to be merged from **spatial_quasi_dynamic_plan.md** or **spatial_dynamic_rupture_plan.md** until the corresponding phase here is GREEN. Cross-reference table:

Driver-plan Phase	Test-plan Phase that gates it
QD §1 (SpatialFrictionConfig, Resolver)	Phases 1, 2
QD §2 (Velocity bundle)	Phase 3
QD §3 (Stress bundle)	Phase 3
QD §4 (driver main)	Phases 7, 8, 9
QD §5 (operator accessors)	Phase 4
QD §6 (sbatch)	Phases 10, 11 must show GREEN before sbatch submission
DYN §0 (LSW schema)	Phases 1, 2
DYN §1 (Fault flux setup)	Phase 5
DYN §2 (Per-QP impedance)	Phase 5
DYN §3 (Bundle reuse)	Phase 3
DYN §4 (driver main)	Phases 7, 8, 9
DYN §5 (operator accessors)	Phase 4
DYN §6 (sbatch)	Phases 10, 11 must show GREEN before sbatch submission
heterogeneous_material_plan.md Phases 1-3	Phase 6

Risk Assessment

High-confidence (low risk)

- Phase 0 fixtures. Generating tiny synthetic meshes + HDF5 sidecars is standard tooling (gmsh + h5py). The committed binaries make tests reproducible without rebuild.
- Phase 1-5 unit tests. Each pins one well-scoped contract; ExpectAbortWithSubstring is a well-understood pattern.

Medium-risk

- Phase 6 GodunovFluxPool memory budget. The 2 GB ceiling is per heterogeneous_material_plan.md R-011. On the mini fixture (~200 tets, 2 layers) it should be well under, but the test is essentially a tripwire; if the SAFS production mesh hits the ceiling, the heterogeneous_material plan's mitigation must be applied before any cluster run.
- Phase 9 end-to-end on **np=4** laptop. Some MPI installations on laptops are flaky; mitigation = SKIP path with clear message + the gate report's (reduced gate) tag.

Low-confidence (need investigation before implementing)

- Phase 8 dyn_v1_wrong_driver_tag_aborts (T_8_13) depends on the schema-extension being implemented correctly in DYN §4. If the field name or default value drifts during driver implementation, this test must be updated alongside.
- Per-QP impedance test (T_5_22) numeric expectation: the impedance ratio between shallow / deep DOFs is $\sqrt{\mu_{\text{shallow}} * \rho_{\text{shallow}} / (\mu_{\text{deep}} * \rho_{\text{deep}})}$. With the 2-layer fixture ($V_s_{\text{shallow}}=1500$, $V_s_{\text{deep}}=3500$, constant ρ), expected ratio = $1500 / 3500 \approx 0.4286$. Verify this empirically against the synthetic sidecar's exact voxel values when implementing the test — small offsets from voxel-edge effects may need a wider tolerance (~5%).
- Fork-based abort tests on macOS 26. macOS occasionally signals SIGABRT through a different exit code than Linux. The ExpectAbortWithSubstring helper must handle both; if neither WIFSIGNALED nor a non-zero exit code is observed when the abort runs, the helper SKIPS with a documented message rather than failing.

Known tricky areas in existing code

- **bp2_serial_smoke.o** pre-existing compile error documented in REVIEW.md is NOT introduced by this plan. The regression matrix (Phase 11) records the current 19/20 umbrella state as the baseline; this plan does not attempt to fix the pre-existing failures.
 - macOS LC_RPATH dedup macro: every new test binary added in this plan must end its link rule with \$(SEAS_POST_LINK_DEDUP_RPATH) per Makefile R-005 (REVIEW.md). Phase 10 umbrella will fail loudly at first run if a test rule misses this.
 - Heterogeneous-material plan's **EvaLat** lifetime contract: tests must keep MaterialField + DataField3D alive for the operator's lifetime. T_3_6 catches the simple violation; production drivers must extend this lifetime through the entire run (already in QD §4 Detailed Req. 2).
-

Out of Scope for This Plan

1. Performance benchmarks. This plan tests correctness only. A separate timing-regression plan can be drafted once a performance baseline is established on Frontera.
2. Frontera-side gates. The cluster sbatch produces its own `verify_*.py` outputs (per sibling Phase 6); cross-validating those against the local gate is a follow-up.
3. GPU offload tests. The Coefficient path is CPU-only per heterogeneous_material plan; GPU testing follows a future port.
4. Property-based / fuzz testing of the TOML parser. The Phase 1 schema tests are example-based. If the TOML grows substantially, switch to a property-based harness in a follow-up.
5. Cross-platform CI (Linux + macOS in the same pipeline). The gate as specified runs locally on macOS-conda mfem-dev; Linux CI integration is a follow-up.